Rapport Projet transverse I - CheckPoint 1

Projet: système de vente des produits de pâtisserie et gâteaux en ligne

Par: Dariia Zhuravel, Mihaela Satcovschi

Table de matières:

I. Introduction 2
II. Rappel du projet 2
III. Use case 3
IV. Analyse des objectifs (révisés) 4
V. Analyse des exigences (révisés) 5
VI. Schéma conceptuel (entité-relation) de la BDD 7
VII. Schéma logique (phpMyAdmin) de la BDD 8
VIII. Algorithme (pseudo-code) 9
IX. Architecture 13
X. Implémentation client-serveur 14
XI. Récuperation données de test 15
XII. Sprint 16
XIII. Conclusion 17

I. Introduction

Dans cette partie du projet, on se concentre sur la mise en place et l'implémentation des données pour le système de vente en ligne des produits de pâtisserie et des gâteaux d'une petite entreprise.

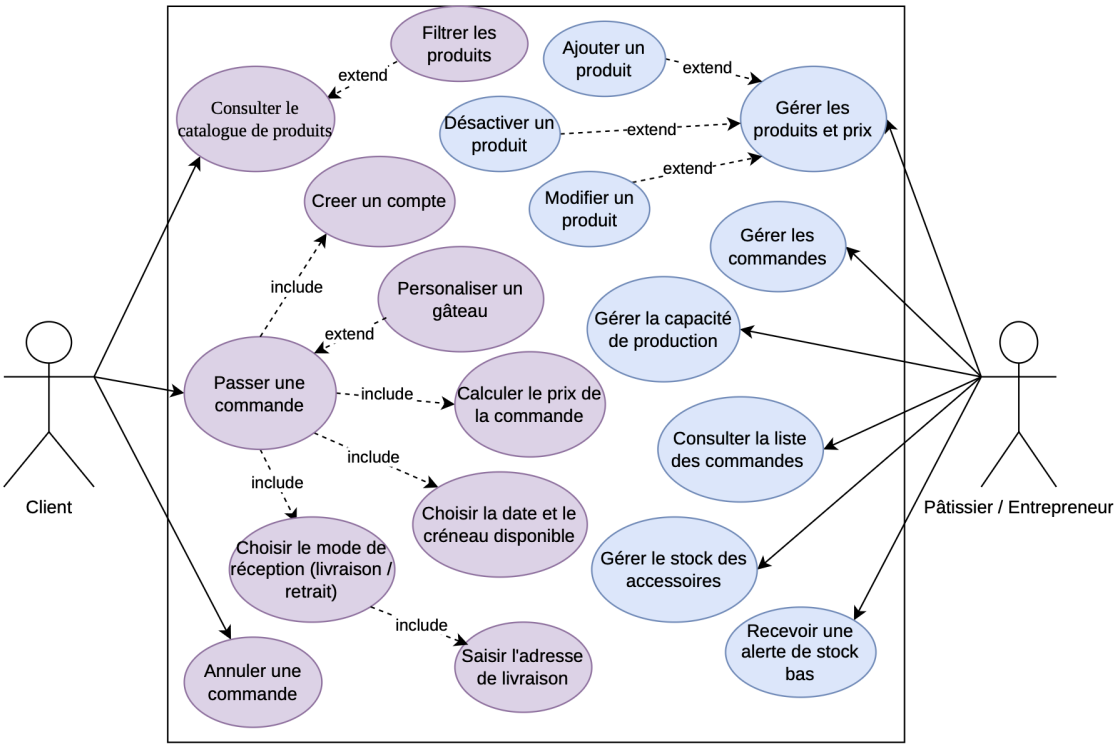
Suite aux retours sur le checkpoint 1 de la part de l'assistant du cours, on a amélioré la construction de la base de données, ainsi que les types de données à insérer dans la base de données. Les autres détails relevants à l'implémentation du projet seront mentionnés dans les sections respectives du checkpoint.

II. Rappel du projet

Dans le contexte d'une petite entreprise, la gestion des commandes se fait souvent de manière informelle (messages, appels, notes), ce qui peut entraîner des erreurs, un manque de visibilité sur les demandes en cours, ainsi que des difficultés à organiser la production et à gérer les stocks (ici, nous nous concentrons spécifiquement sur les produits complémentaires (emballages, bougies, décorations, etc.) et non sur les ingrédients de fabrication). L'objectif de notre système est donc de centraliser ces informations dans une seule plateforme, afin de faciliter la prise de commande, de réduire les oublis et d'améliorer l'organisation au quotidien.

L'idée principale est de créer un site web grâce auquel les clients pourront découvrir la gamme de produits, les designs de base possibles, les garnitures et les prix. Nous souhaitons également ajouter une interface de commande où le client pourra choisir la date qui lui convient, vérifier les créneaux disponibles, personnaliser un gâteau (poids, garniture, design, inscription, extras), ajouter des commentaires à la commande, choisir la livraison ou le retrait et obtenir le calcul du prix. Du côté de l'entrepreneur, une interface d'administration permettra de gérer les produits disponibles, les options, les prix, etc. Il disposera aussi d'une base de données contenant toutes les commandes, les stocks(au moins pour les accessoires) et les produits avec prix.

III. Use case



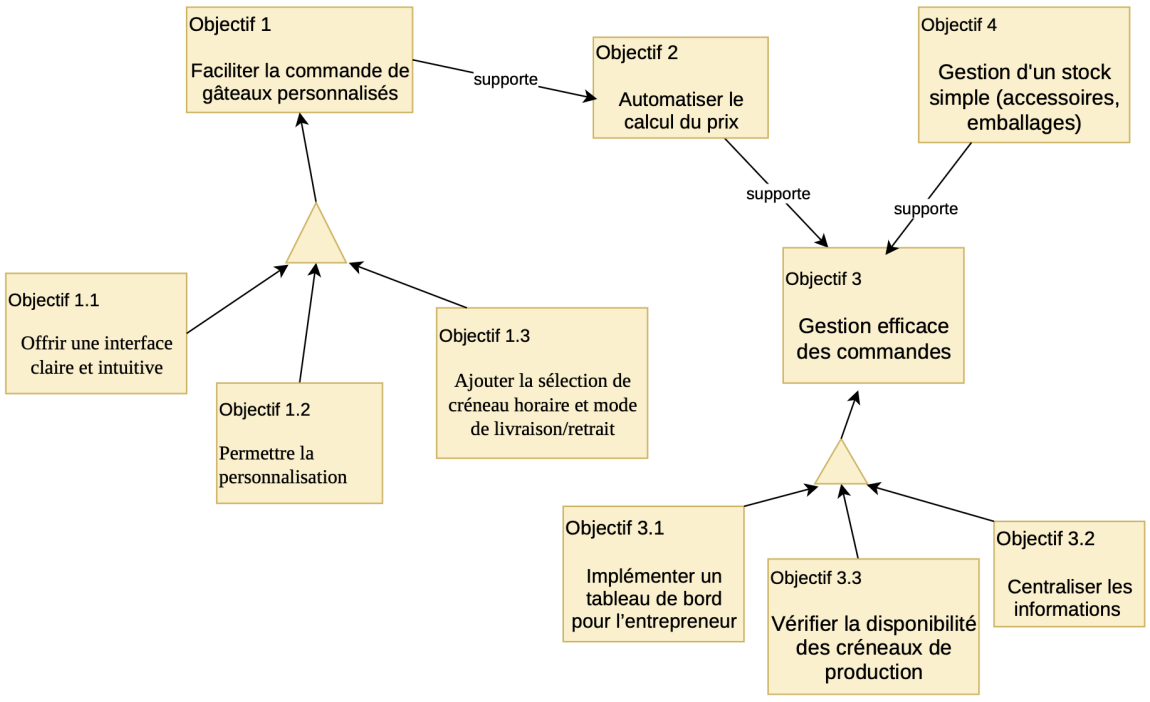
Le diagramme de cas d'utilisation représente les interactions d'un Client et d'un Pâtissier / Entrepreneur dans notre système de commande de pâtisseries en ligne.

Du côté du client, les principales fonctionnalités sont la consultation du catalogue de produits, le passage d'une commande ou l'annulation. De plus, le cas d'utilisation «filtrer les produits» est associé à «consulter le catalogue de produits» par une relation «extend», car le client peut consulter le catalogue sans nécessairement appliquer de filtre.

Le cas d'utilisation « passer une commande » occupe une place centrale dans le diagramme, car plusieurs actions en dépendent directement, notamment le calcul du prix total, le choix d'une date et d'un créneau horaire disponible, ainsi que la sélection du mode de réception, par livraison ou par retrait. Ces actions sont représentées comme liées au processus principal par une relation « include », puisqu'elles sont nécessaires à la finalisation de la commande. La création de compte est également liée à ce processus par une relation « include », dans la mesure où, dans notre système, elle est considérée comme nécessaire pour faciliter la gestion et le suivi des commandes par l'utilisateur. En revanche, la personnalisation d'un gâteau correspond à une possibilité supplémentaire représentée par une relation « extend », car elle n'intervient pas dans toutes les commandes, mais seulement lorsque le client souhaite adapter le produit à ses besoins.

Du côté du pâtissier ou entrepreneur, le système offre des fonctions de gestion avancée de l'offre et des opérations. Il peut gérer les produits et leurs prix, et les cas d'utilisation « ajouter un produit », « modifier un produit » et « désactiver un produit » sont reliés à cette fonction par des relations « extend », puisqu'ils correspondent à des actions spécifiques réalisées selon le besoin, et non à des étapes systématiques de chaque gestion. Le professionnel dispose aussi d'une gestion des commandes, incluant la consultation de la liste des commandes et l'annulation d'une commande si nécessaire. Dans ce cadre, « annuler une commande » peut également être représenté par une relation « extend », car cette action n'intervient que dans des situations particulières et ne fait pas partie du déroulement normal de chaque gestion de commande. Enfin, il gère le stock des accessoires, la capacité de production, et reçoit des alertes en cas de stock bas, ce qui lui permet d'anticiper les besoins et de garantir la disponibilité des produits pour les clients.

IV. Analyse des objectifs (révisés)



Ce schéma présente des objectifs de notre système de commande de gâteaux personnalisés. L'Objectif 1 vise à faciliter la commande de gâteaux personnalisés en mettant l'accent sur l'expérience client : il s'agit d'offrir une interface claire et

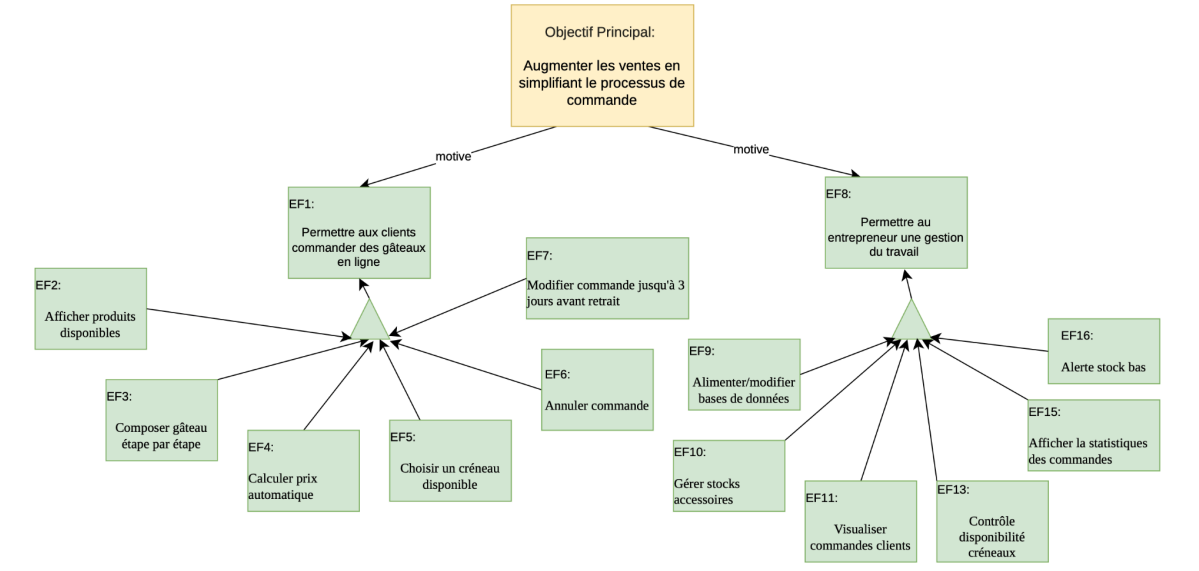
intuitive (1.1), de permettre la personnalisation des produits (1.2) ainsi que d'ajouter la sélection du créneau horaire et du mode de livraison ou de retrait (1.3).

L'Objectif 2 concerne l'automatisation du calcul du prix, ce qui vient soutenir la gestion des commandes en rendant la tarification plus rapide, cohérente et fiable.

L'Objectif 3 porte sur une gestion efficace des commandes avec trois sous-objectifs complémentaires : implémenter un tableau de bord pour l'entrepreneur (3.1), centraliser les informations (3.2) et vérifier la disponibilité des créneaux de production (3.3) afin d'aligner les demandes clients avec les capacités de production.

Enfin, l'Objectif 4 prévoit la gestion d'un stock simple d'accessoires et d'emballages, ce qui vient également soutenir le bon déroulement du processus de gestion.

V. Analyse des exigences (révisés)



Ce schéma décrit les principales exigences fonctionnelles de notre application, à partir d'un objectif principal: augmenter les ventes en simplifiant le processus de commande. Pour cela, nous prévoyons de développer une application permettant à la fois aux clients de commander des gâteaux en ligne (EF1) et à l'entrepreneur de gérer efficacement son travail (EF8).

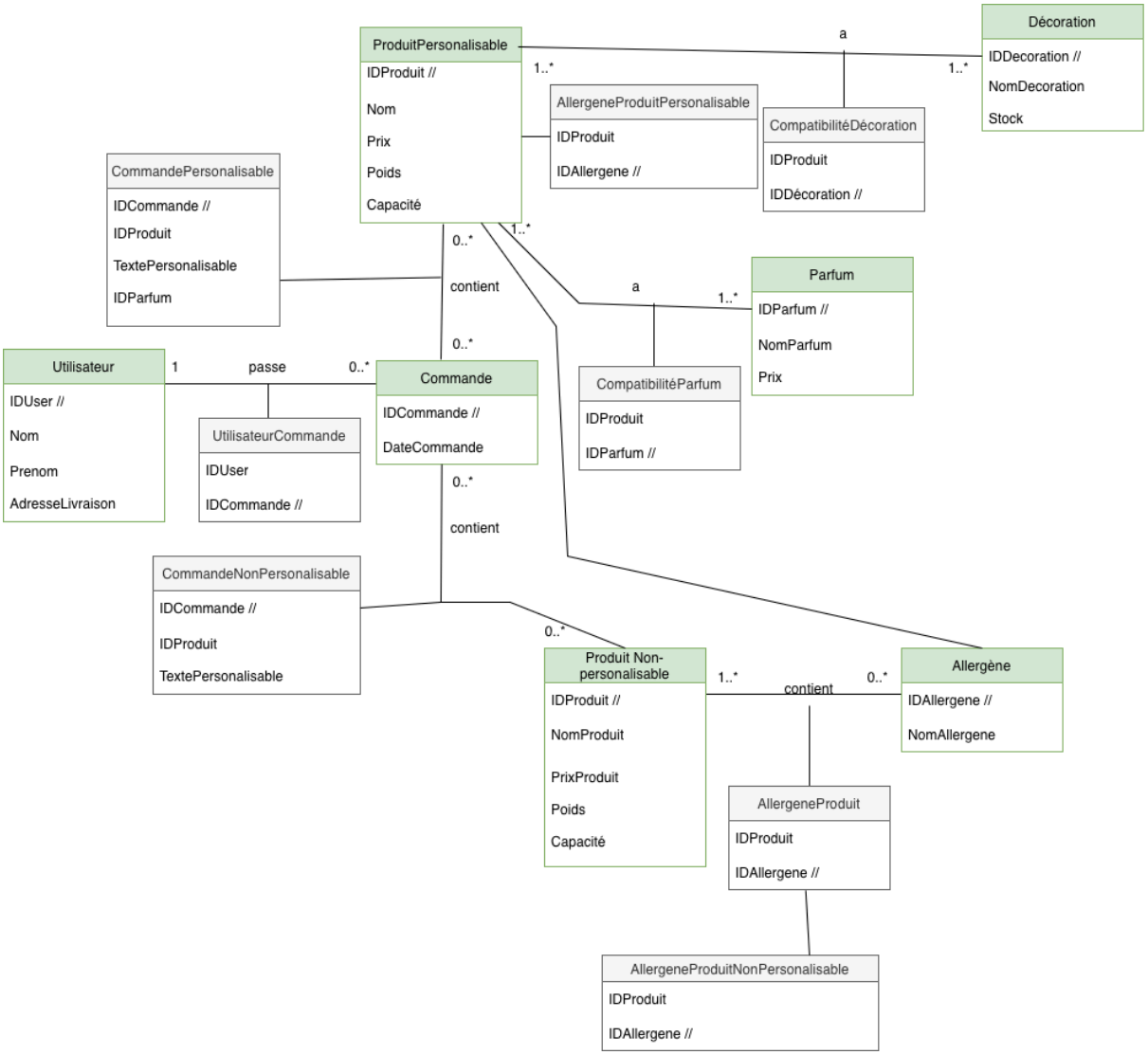
Autour de cet objectif central, plusieurs fonctions orientées client sont mises en avant : afficher les produits disponibles (EF2), composer un gâteau étape par étape (EF3), calculer automatiquement le prix (EF4), choisir un créneau disponible (EF5), annuler une commande (EF6) ou encore la modifier jusqu'à trois jours avant le retrait (EF7), ainsi que visualiser les commandes clients (EF11).

Du côté de l'entrepreneur, le schéma prévoit des fonctionnalités de gestion interne telles qu'alimenter ou modifier les bases de données (EF9), gérer les stocks d'accessoires (EF10), déclencher une alerte de stock bas (EF16), afficher des statistiques sur les commandes (EF15) et contrôler la disponibilité des créneaux (EF13), ce qui garantit une exploitation plus structurée et efficace de l'activité.

En complément de ces aspects fonctionnels, on a défini des exigences non-fonctionnelles qui garantissent la qualité globale de l'application, pour le client comme pour l'entrepreneur :

- Utilisabilité : l'application doit être simple à utiliser et lisible sur ordinateur et mobile.
- Sécurité : accès protégé par mot de passe, droits séparés client/admin, validation des champs saisis.
- Fiabilité : l'application ne doit pas planter en cas d'erreur ; elle affiche des messages clairs si une action échoue.
- Performance : les pages principales doivent se charger rapidement en utilisation normale.

VI. Schéma conceptuel (entité-relation) de la BDD



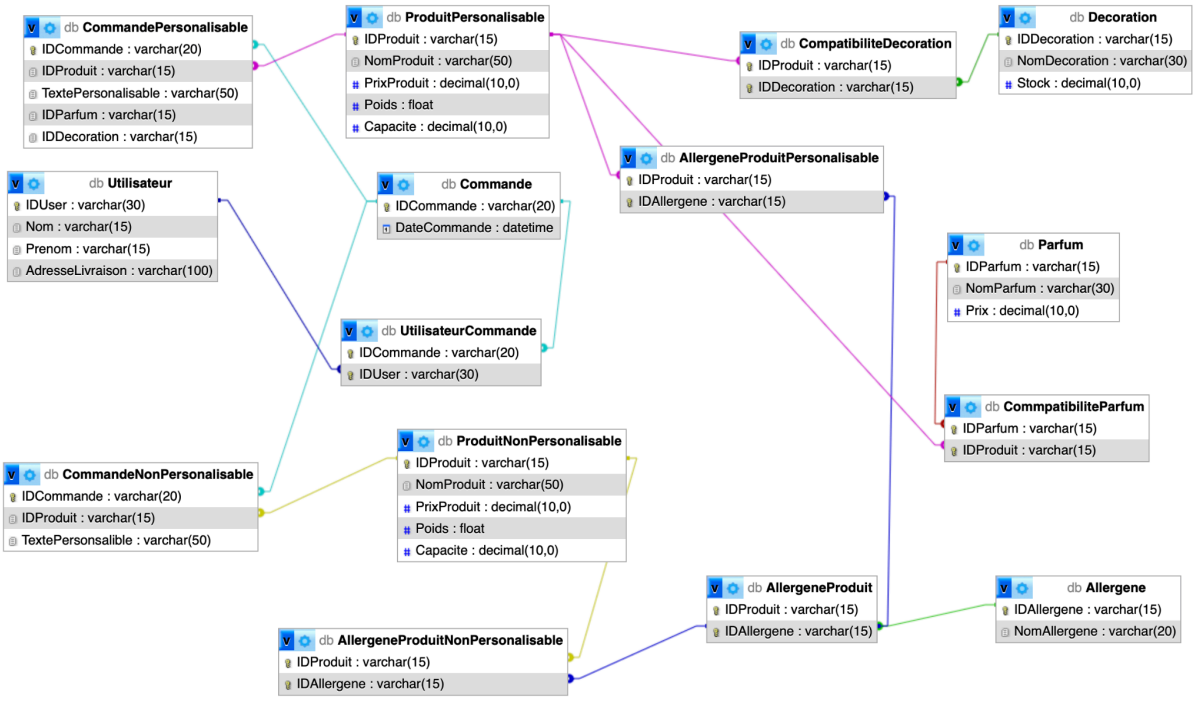
Suite à un rendez-vous avec l'assistant, on a refait la structure de notre base de données afin d'avoir une structure correcte, logique et fonctionnelle. Cette version ne correspond pas à la version du CheckPoint 1, donc tous les détails de la base de données (création et tables existantes), vont être détaillés dans le point suivant, concernant le schéma logique.

On a également fait quelques modifications qui peuvent être considérées comme redondantes, mais elles étaient faites pour la partie de récupération de données par le programme. On parle ici des tables Commandes, CommandeNonPersonalisable et CommandePersonalisable. La raison pour laquelle ces tables sont toutes séparées et pas mises dans une grande table est parce que si la table Commande serait construite d'utiliser comme clé étrangère l'attribut "IDProduit" de la table ProduitPersonalisable

et l'attribut "IDProduit" de la table "ProduitNonPersonalisable", alors pour une commande qui ne contient qu'un type de produit (produit personalisable ou produit non-personalisable), le tuple devrait être NULL pour le type de produit qui ne serait pas présent, et dans la lecture de la base de données avec python, les tuples de la base de données ne seraient pas clairement comprises et traitées. C'est bien pour ça qu'on a créé la table Commande avec uniquement les attributs "IDCommande" et "DateCommande" (-> par cette pratique on ne lie plus la table Commande aux 2 tables de produits) et les tables CommandePersonalisable et CommandeNonPersonalisable où dans les 2 cas, l'attribut "IDProduit" provient d'une seule table uniquement.

Par le même raisonnement, on a modifié la table AllergeneProduit, qui était dans la même construction: l'attribut "IDProduit" provient des 2 tables. Par conséquent, on a divisé cette table en 2 autres tables supplémentaires, conformes aux type de produit avec les allergènes concernés.

VII. Schéma logique (phpMyAdmin) de la BDD



Voici le screenshot de l'écran du schéma logique depuis phpMyAdmin.

VIII. Algorithme (pseudo-code)

On a considéré que ça serait le plus important et le plus pertinent pour notre système en cours de développement d'implémenter l'algorithme concernant l'établissement de statistiques diverses (e.g. produit le plus populaire durant la dernière semaine, tendances). Plus précisément, en ce qui concerne les produits de pâtisserie offerts par le site (Produits Personnalisables ou Produits Non-Personnalisables), on aimerait ajouter une étiquette, sur le menu des produits, ainsi que sur le produit concerné que le produit donné soit un "Bestseller". Une autre application de cet algorithme serait, dans le cadre de la personnalisation d'un gâteau par le client, le moment de la création d'une commande, lors de la sélection d'un parfum pour le gâteau parmi la liste des variantes possibles, voir également si un des parfums est un choix populaire parmi les clients du système.

Le document du cours concernant la liste des projets, propose également l'algorithme tel que alerte lorsque le stock diminue, mais comme on l'a mentionné lors du rapport Checkpoint 1, notre système travaille uniquement avec le traçage et gestion du stock des décorations, car on considère que d'un point de vue business, ça serait inutile et gaspillant de faire la gestion d'un parfum, par exemple, qui pourrait expirer jusqu'à ce qu`un client le choisit en tant que parfum choisi pour le produit de sa commande. Donc, par ce raisonnement, on a décidé de ne pas appliquer un tel algorithme pour notre système et se concentrer uniquement sur l'algorithme concernant les statistiques des produits et parfums populaires, tel que observés lors des commandes effectuées.

```
INITIALIZE empty dictionary sales_count

FOR each order IN orders:

        product_id = order.product_id


         IF product_id not in sales_count:

                sales_count[product_id] = 0

        sales_count[product_id] = sales_count[product_id] + 1

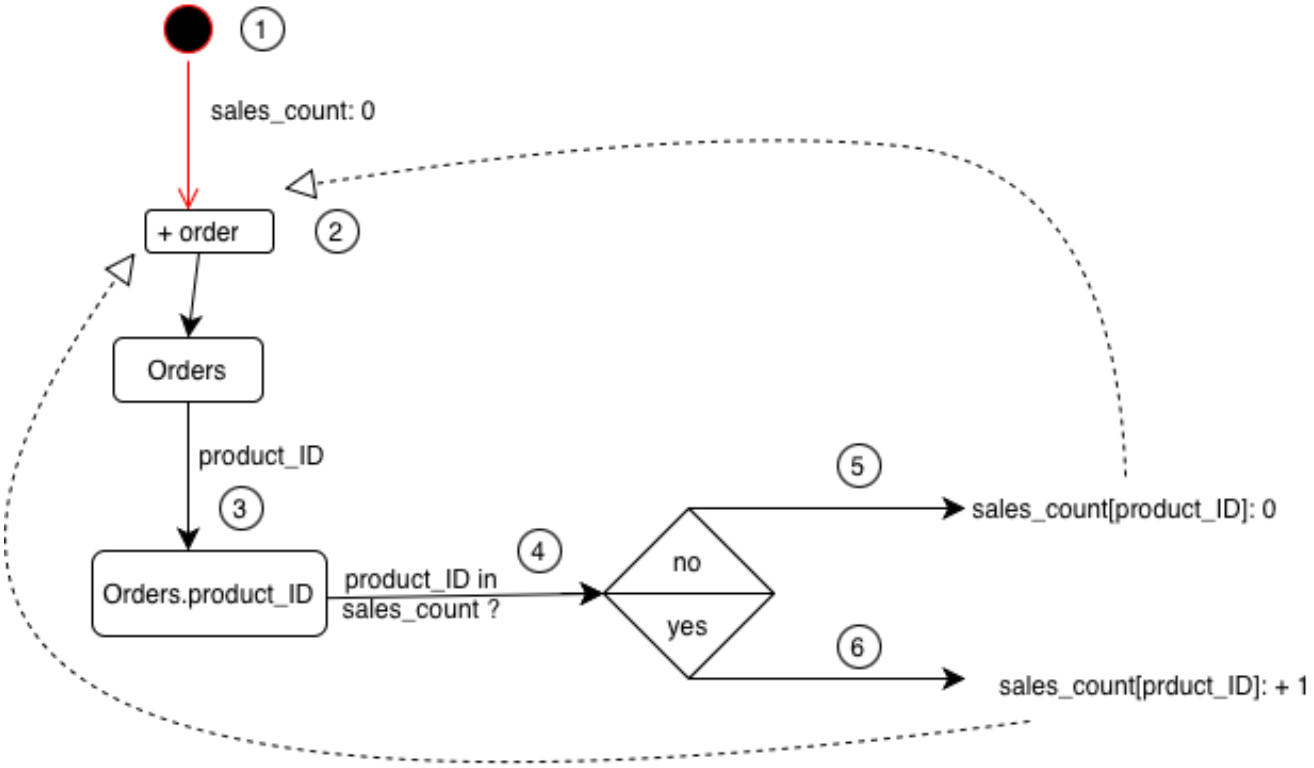

SORT sales_count by value (descending)
```

bestsellers = first 5 products in sorted list

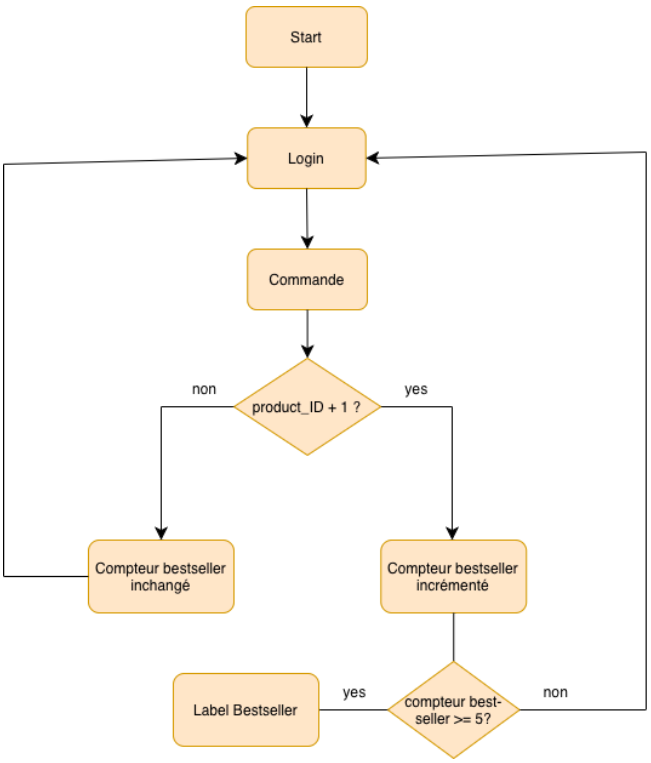
FOR each product IN bestsellers:

tag product as "bestseller"

Diagramme UML correspondant:



Flowchart concernant:



Voici le pseudo-code l'algorithme calculant les produits bestsellers (on va utiliser la convention que pour qu'un produit soit un bestseller, il faut que au moins 5 personnes l'ont commandé).

Ensuite l'algorithme pour établir le parfum bestseller:

INITIALIZE empty dictionary sales_count

FOR each order IN orders:

parfum_id = order.parfum_id

IF parfum_id not in sales_count:

sales_count[parfum_id] = 0

sales_count[parfum_id] = sales_count[parfum_id] + 1

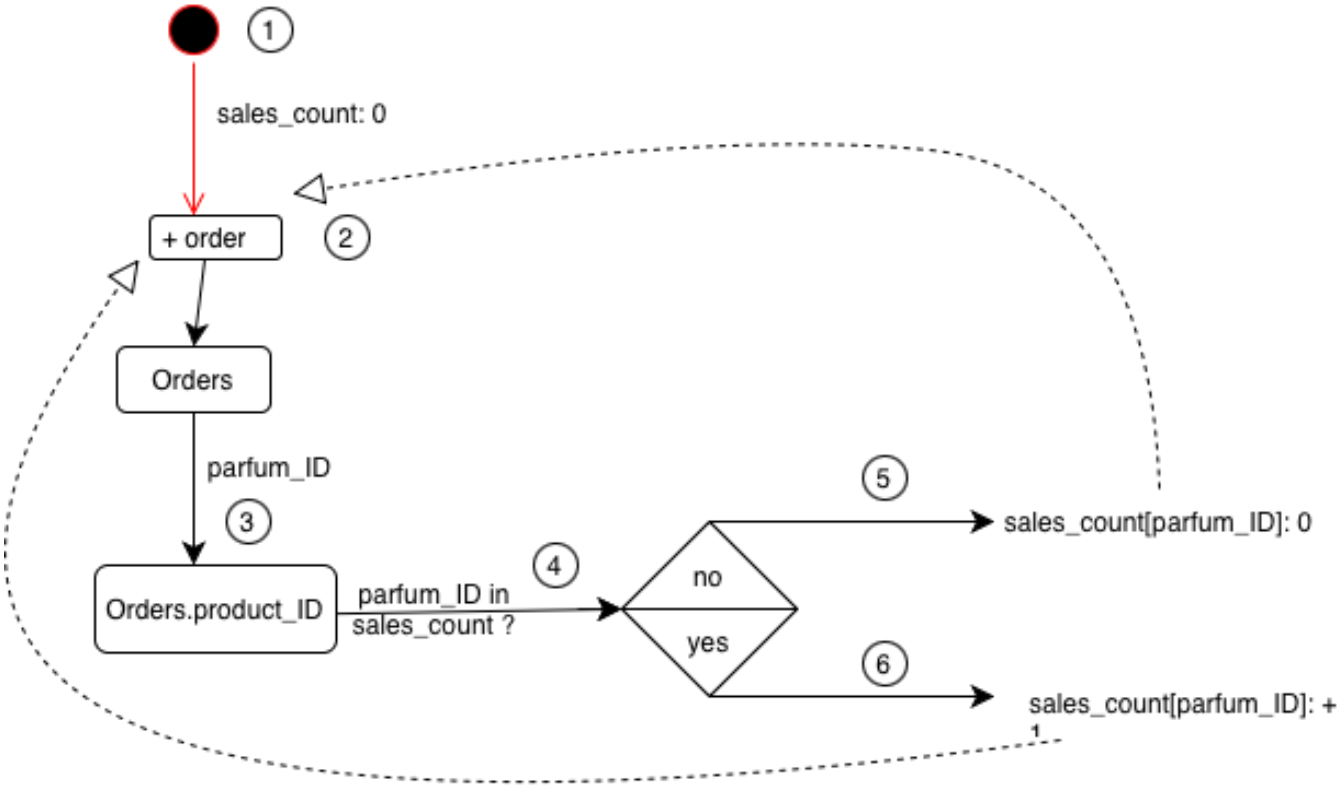
SORT sales_count by value (descending)

bestsellers = first 5 parfums in sorted list

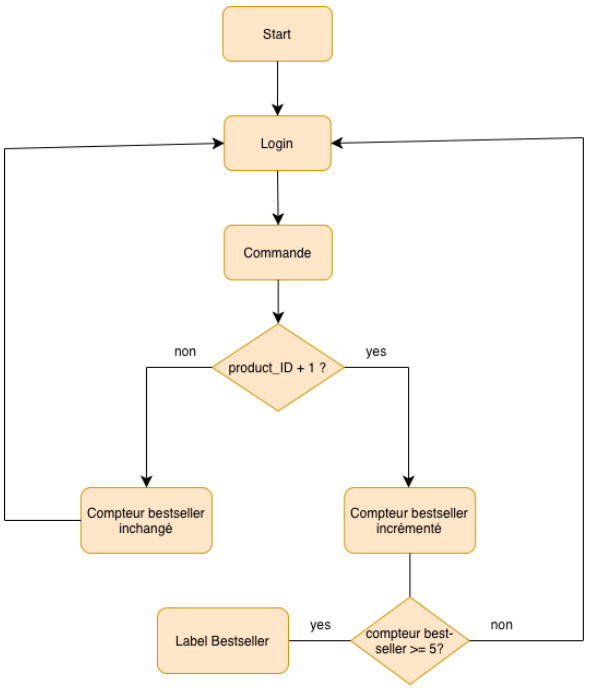
FOR each parfum IN bestsellers:

tag product as "bestseller"

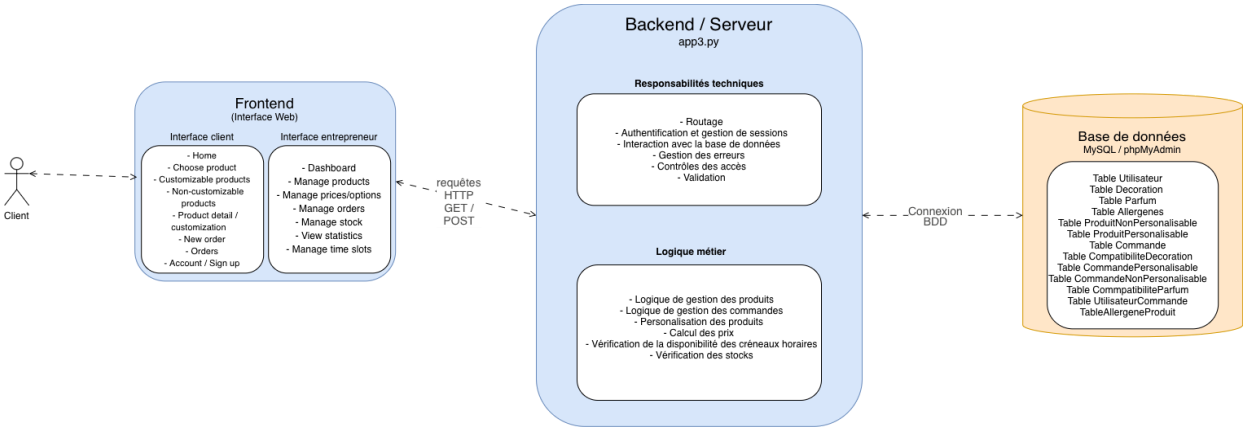
Diagramme UML correspondant;



Flowchart correspondant:



IX. Architecture



Voici ce qu'on envisage comme détails de notre architecture, avec des parties détails des fonctionnalités de la partie frontend du serveur, de la partie backend et de la base de données.

Notre application est organisée en trois grandes parties : le frontend, le backend et la base de données. Cette séparation permet de mieux structurer le système : le frontend gère l'affichage et les actions de l'utilisateur, le backend traite les requêtes et applique la logique de l'application, et la base de données stocke toutes les informations nécessaires.

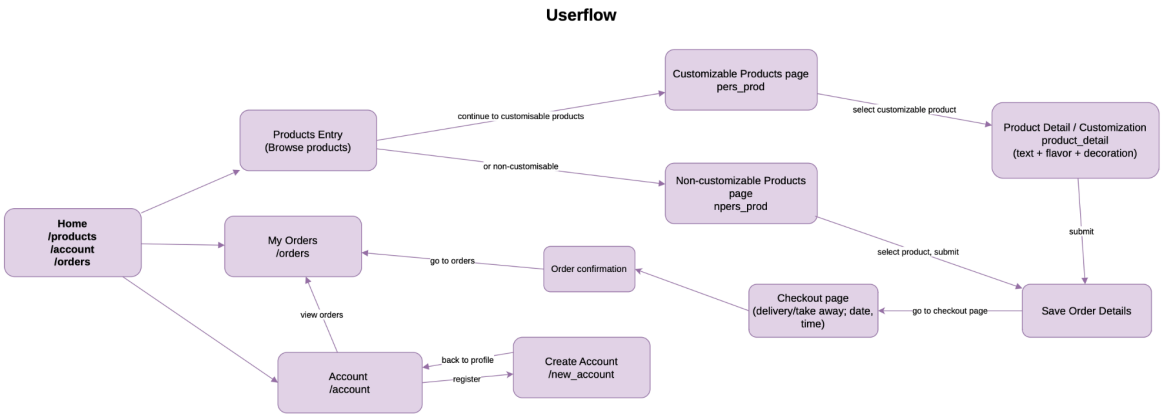
Le frontend correspond à l'interface web. On distingue une partie client, qui permet de consulter les produits, voir les détails, personnaliser certains produits, passer une commande et gérer son compte, et une partie entrepreneur, qui sert à administrer la plateforme. Dans cette interface administrateur, on peut gérer les produits, les prix, les commandes, les stocks, les statistiques et les créneaux horaires.

Le backend, représenté ici par le fichier *app3.py*, joue le rôle d'intermédiaire entre l'interface et la base de données. Il reçoit les requêtes envoyées par le frontend en HTTP (GET/POST), les traite, vérifie que les données sont correctes, gère l'authentification, les sessions, les accès et les erreurs, puis renvoie une réponse adaptée.

Cette partie contient aussi la logique métier de l'application. C'est là que sont gérées les commandes, les produits, la personnalisation, le calcul des prix, la vérification des stocks et la disponibilité des créneaux. En d'autres mots, c'est le backend qui décide si une action demandée par l'utilisateur est possible ou non.

Enfin, la base de données MySQL / phpMyAdmin permet de stocker toutes les données du système. On y retrouve par exemple les tables des utilisateurs, des produits, des commandes, des parfums, des décorations et des allergènes. Certaines tables servent aussi à gérer les relations entre les éléments, par exemple entre un utilisateur et une commande, ou entre un produit et ses allergènes.

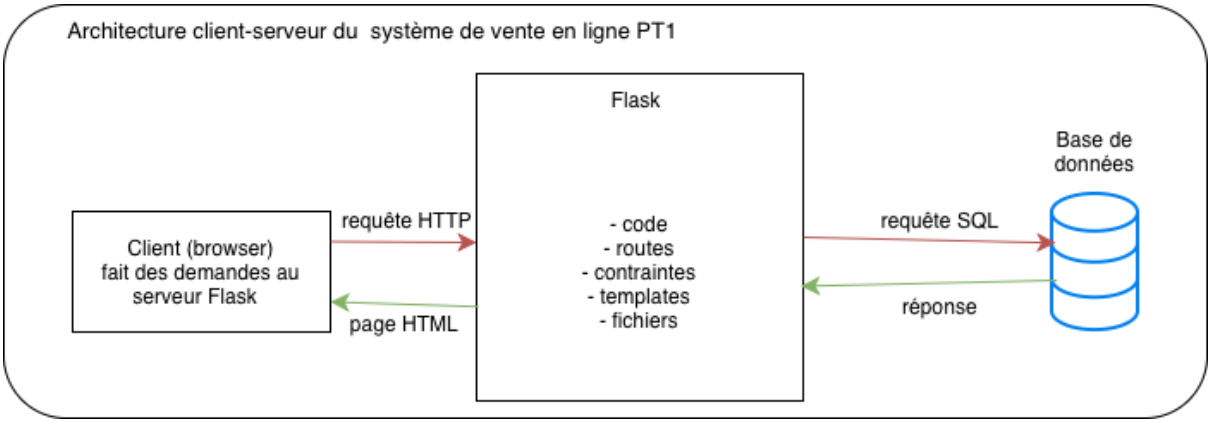
On a également créé le schéma représentant de comment toutes les pages de la parties front-end (les fichiers .html) sont reliés entre eux via les routes créés en backend avec Flask (le grand fichier .py)



X. Implémentation client-serveur

Le système fonctionnel de vente en ligne a une implémentation client-serveur, où le client est représenté par le la session qui se passe dans le web browser, qui elle, la session, demande du serveur qui est l'application Flask, qui lui donne informations et utilise la base de données derrière.

Plus clairement, quand un client (une fois connecté à son compte de l'application), le browser dans lequel il se retrouve le client va envoyer une requête HTTP vers le serveur Flask (qui représente notre service mis en place), le serveur Flask va analyser la requête, surtout en exécutant des requêtes SQL et en final, le serveur Flask va retourner une page HTML avec les données collectées selon la requête du client.

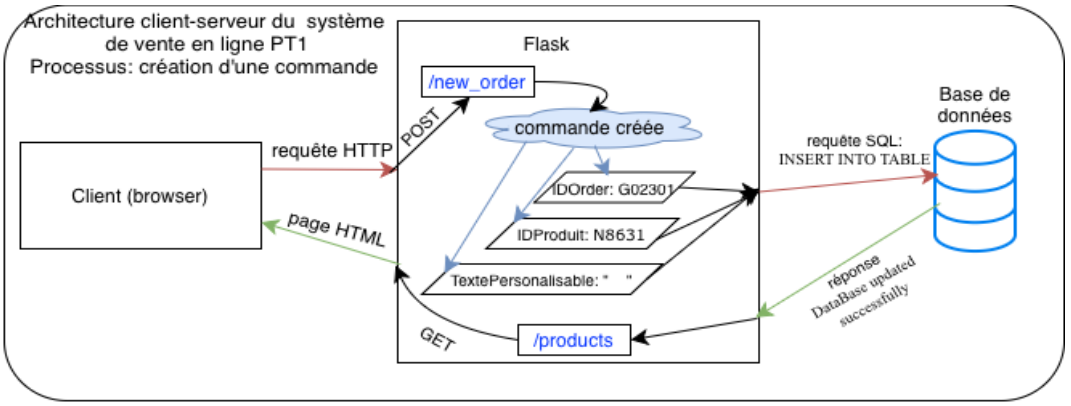


Cette implémentation est extrêmement importante pour le bon fonctionnement du code et pour une bonne et pratique utilisation de la base de données par les utilisateurs. Toute action entreprise via le système de vente ne serait pas possible sans le code derrière et les bonnes redirections d' URL.

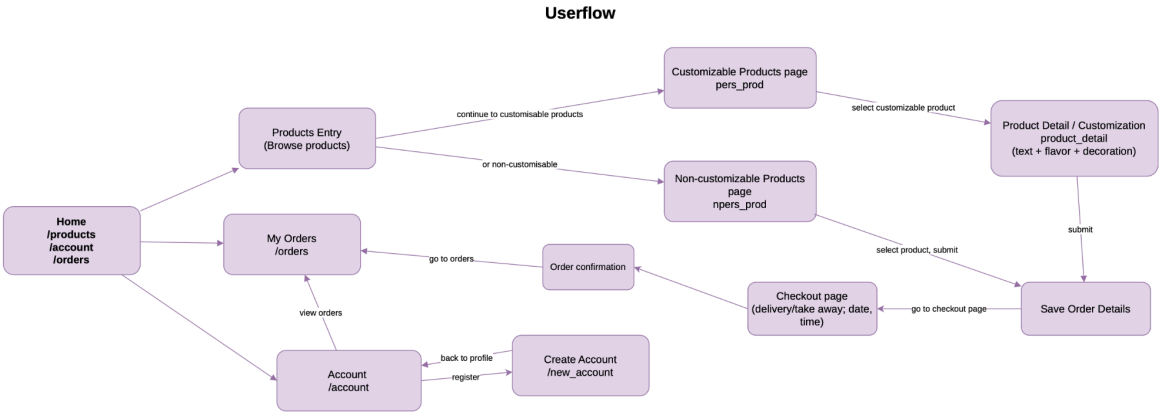
Pour montrer à quoi ressemble un bon fonctionnement de l'implémentation client-serveur, on illustre avec l'exemple de la création d'une commande par un utilisateur. Lorsqu'un client clique sur "Créer une nouvelle commande", le browser va envoyer une requête de type POST vers la route /new_order. Pendant que la commande est créée, le serveur Flask crée et propage un ID Order qui va servir comme numéro de commande unique dans la table Commande et dans la table CommandePersonalisable ou CommandeNonPersonalisable, selon le type et les détails de la commande (IDProduit, TextePersonalisé, potentiellement IDParfum, IDDecoration). Le processus est fini et le client est redirigé vers le catalogue de produits (une fois

la commande confirmée) et toutes les tuples concernant sa commande sont actualisées.

Visuellement, le processus ressemble à:



XI. Récuperation données de test



Avec ce diagramme qu'on a antérieurement présenté, on observé le user flow exacte, avec les chemins parcourus d'un user lors de la création d'une commande, par exemple.

Un utilisateur va se retrouver initialement sur le route "/" ou "/home". Les URLs disponibles depuis "/" sont "/products", "/account" et "/orders".
En choisissant d'aller sur la route "/account", il va pouvoir soit se connecter, soit créer son compte. Ce processus est déterminé par le fait que l'utilisateur va mettre son username et appuyer sur Login, et la base de données va vérifier si cet username existe ou pas encore dans la base de données. Si son username existe, il va être redirectionné vers la route "/profile" et sinon, il va être invité à créer son compte en étant redirectionné vers la route " /new_account".

Une fois ce processus d'authentification fini, l'utilisateur va visualiser, depuis son profil, les commandes qu'il a effectué si c'est le cas et il va être invité à créer une nouvelle commande.

Il sera ensuite redirectionné vers la route "/products", d`où l'utilisateur va choisir la catégorie de produit qu'il aimerait acheter: Produit Personalisable (rédirection vers route "/personalisable_products") ou Produit Non-personalisable (rédirection vers route "/nonpersonalisable_products").

Une fois son produit séléctionné, il va pouvoir le modifier et personaliser (rédirection vers route "/product/<id_product>") et une fois confirmé, l'utilisateur va être rédirectioné vers la route "/new_order" et "/orders".

XII. Sprint

sprint 3: établissement des décisions concernant les détails fonctionnels, division des tâches pour le rapport, établissement détails des chemins et designs backend et frontend

sprint 4: changement des certains choix des chemins et designs backend et frontend, partage des idées de complétion du rapport

XIII. Conclusion

On considère que jusqu'à présent on a bien avancé avec la création du système et l'implémentation de la base de données dans le système. On observe à présent les parties bien fonctionnelles de notre système dans son stade actuel, mais aussi des points d'amélioration, ainsi que des nouvelles idées à ajouter et à mettre en place dans le cadre du système qui pourrait le rendre encore plus utile. On parle ici de l'initiative de créer un mode "Entrepreneur" qui va pouvoir gérer les utilisateurs, commandes, stocks des décorations et catalogues des produits via l'interface du site.